

IUP MIAGE – MOD 16 Systèmes Ouverts et Distribués

Chapitre III :

Les RPC

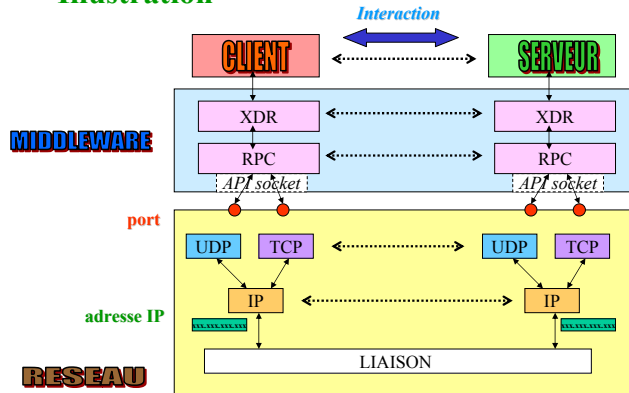
Appels de Procédure à Distance

RPCs : caractéristiques générales (1/2)

- **Une interface**
 - **RPC** = REMOTE PROCEDURE CALL
 - interface entre les programmes d'application et les protocoles de communication de plus haut niveau que les sockets
 - acteur principal : SUN (UNIX/TCP-IP)
- **Une abstraction**
 - rendre transparent le fait qu'une procédure s'exécute sur un site distant et dans un environnement éventuellement différent : masquer au programmeur les aspects de communication et les différences de représentation de l'information
- **Des primitives**
 - attribuer aux processus un rôle de client ou de serveur
 - échanger des messages protocolaires en rendant transparents les services de communication réseau
 - coder/décoder les informations échangées

Les RPCs : caractéristiques générales (2/2)

• Illustration



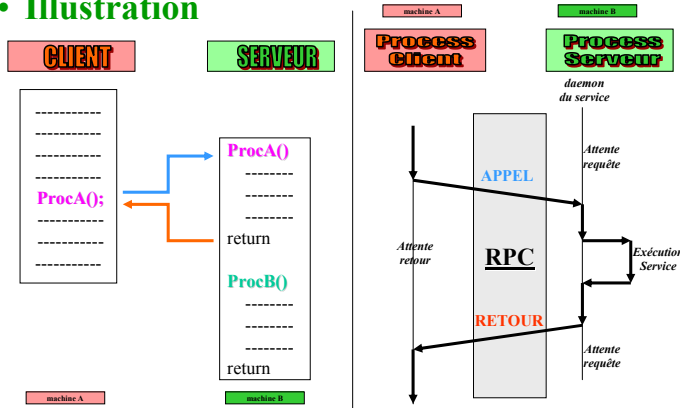
Les RPCs : le modèle (1/3)

• Mise en oeuvre du modèle Client/Serveur

- **requête (Client - -> Serveur)**
= appel de la procédure distante
les paramètres d'exécution sont communiqués au serveur
- **réponse (Serveur - -> Client)**
= retour de l'appel de la procédure distante
les résultats sont communiqués au client
- **synchronisation**
= mode d'interaction synchrone
l'appelant (le client) attend le retour de l'appel avant de pouvoir exécuter l'instruction suivante.

Les RPCs : le modèle (2/3)

• Illustration



Les RPCs : le modèle (3/3)

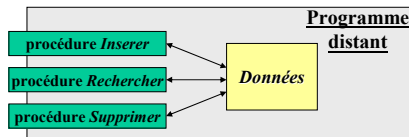
• Différences entre appels locaux et distants

- temps d'acheminement des informations échangées peuvent augmenter les temps d'exécution
- pas d'argument de type pointeur...
- les descripteurs des fichiers d'E/S ne sont pas accessibles aux procédures distantes
- possibilités de défaillances (client, serveur)
- problèmes de sécurité (authentification, confidentialité)
- aide de XDR pour la représentation commune des données

Les RPCs : le programme distant (1/4)

• Concept de programme distant

- unité s'exécutant sur une machine distante ($\cong$ serveur)
- programme distant = ensemble de procédures et de données propres :



• Gestion des versions

- Dès qu'une procédure est ajoutée, modifiée (paramètres), ou supprimée :
==> nouvelle version du programme distant

Les RPCs : le programme distant (2/4)

• Identification RPC

- Chaque programme distant est identifié par un numéro entier unique sur 32 bits (entre 0x20000000 et 0x3FFFFFFF pour les utilisateurs)
 - A chaque programme est également associé un numéro de version (évolution aisée sans obligation de re-numérotation)
 - Les procédures d'un programme distant sont identifiées séquentiellement par les entiers 1, 2, ..., N
- ==> Toute procédure distante est identifiée par un triplet
(n° programme, n° version, n° procédure)
- ==> Moyen d'identification pour le client appelant

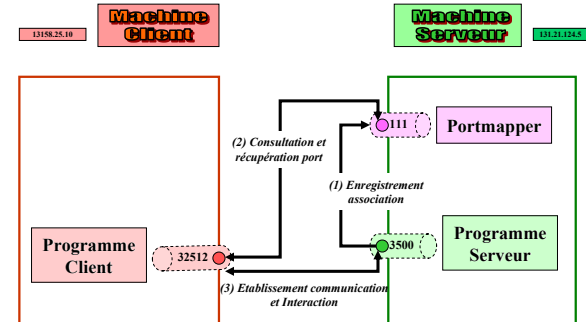
Les RPCs : le programme distant (3/4)

• Localisation RPC

- L'identification d'une procédure ne sous-entend pas sa localisation
 - Conformément au monde TCP/IP, le processus serveur doit être localisé grâce au doublet (@sse IP, #port)
- => correspondance biunivoque identificateur de programme (32 bits) / #port (16 bits) impossible de façon directe
- **Solution** : intervention d'un intermédiaire qui enregistre dynamiquement la correspondance entre programme distant et numéro de port et la fait connaître au client lors d'une invocation
- => Notion de port mapper (daemons `portmap` ou `rpcbind` ss UNIX)

Les RPCs : le programme distant (4/4)

• Le Port Mapper : principe



Les RPCs : XDR (1/2)

• Rôle de XDR (Sun)

- **XDR (eXternal Data Representation)** : protocole permettant de définir une représentation standard des données échangées entre machines client et serveur
 - fonctionnalité de la couche Présentation (ISO/OSI)
 - représentation des messages RPC, des paramètres
- => interprétation commune
- exemple : entier de 32 bits ayant la valeur 260
 - 0014 (big endian)
 - 4100 (little endian)
- Contrairement à ASN-1, l'encodage XDR ne porte que sur les données représentées mais pas sur leur type
- => les interlocuteurs doivent s'entendre sur le format exact des données échangées

Les RPCs : XDR (2/2)

• L'API XDR

- Bibliothèque de fonctions de conversion XDR (interopérabilité)
 - Manipulation du type XDR : flot
- fonction permettant sa gestion (création, décodage/décodage, taille...)
- ```

- XDR *xdrs; /* Émetteur */
char Flot_out[TAILLE_FLOT];
...
xdr_mem_create(xdrs, Flot_out, TAILLE_FLOT, XDR_ENCODE);
...

```
- **sérialisation** : ajouter des données en les codant dans le flot une fonction par type : xdr\_NomDuType (`xdr_int`, `xdr_long`, `xdr_float`, `xdr-string`, `xdr_opaque`, ...)
- ```

- int i = 260;
...
xdr_int(xdrs,&i);
  
```
- **désérialisation** : opération inverse extraction et décodage
- ```

- XDR *xdrs; /* Récepteur */
...
xdr_mem_create(xdrs, Flot_out, TAILLE_FLOT, XDR_DECODE);
xdr_int(xdrs,&j);

```

## Les RPCs : format des messages (1/1)

### • Format des messages RPC

- Messages supports du protocole RPC
- Longueur variable car le nombre d'arguments de la procédure appelée ne peut être déterminé à l'avance

|                  |
|------------------|
| ID Message       |
| Type Message     |
| Version RPC      |
| N° Programme     |
| N° Version       |
| N° Procédure     |
| Authentification |
| Arguments        |

## Les RPCs : Les Talons (1/2)

### • Assurer la transparence...

- La notion de distance dans les RPCs implique l'adjonction de code entre l'appel de la procédure et la procédure appelée
  - **côté client** : ce code doit
    - » encoder les arguments (xdr)
    - » créer un message de type RPC CALL
    - » émettre ce message vers le programme distant
    - » attendre les résultats, puis les décoder vers la représentation interne de la machine
  - **côté serveur** : ce code doit
    - » accepter une requête RPC
    - » décoder les arguments vers la représentation interne de la machine
    - » dispatcher le message vers la procédure adéquate
    - » construire la réponse et l'encoder (xdr)
    - » émettre le message RPC REPLY vers la client

## Les RPCs : Les Talons (1/2)

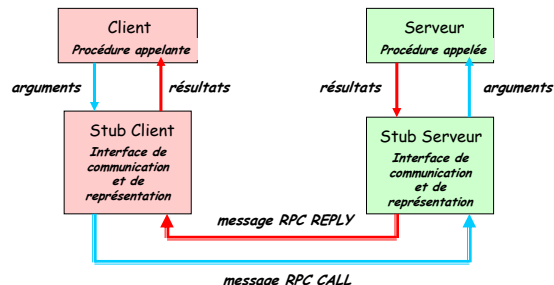
### • Assurer la transparence...

- Mise en œuvre de procédures remplaçant le code conventionnel d'appel et de retour de procédure locale => les talons (stubs)

=> **côté client : stub (talon)**

=> **côté serveur : skeleton (squelette)**

- Illustration :



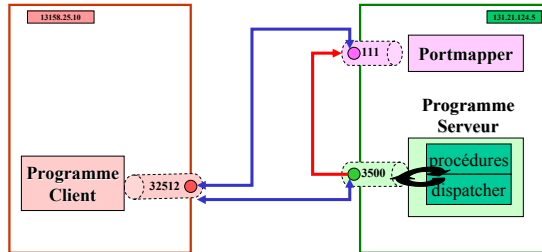
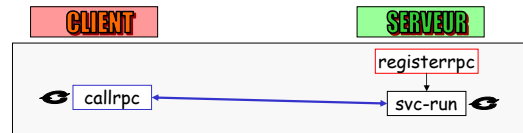
## Les RPCs : interface haut-niveau (1/2)

### • Principe

- Transparence totale de la communication
- Interface réduite à 3 primitives :
  - le **serveur** doit enregistrer son programme et ses numéros auprès du port mapper
    - => **primitive registerrpc** :
  - Puis il se met en attente de requêtes entrantes, se branche sur le code de la procédure demandée (dispatching) et retourne le résultat
    - => **primitive svc\_run** :
- le **client** initie l'interaction par un appel de procédure.
  - => **primitive callrpc** : à partir des numéros indiqués contacte le port mapper de la machine distante du serveur, puis initiation de l'interaction RPC avec le serveur (émission message de requête, réception message réponses)
- utilisation commune d'une routine xdr (paramètre)

## Les RPCs : interface haut-niveau (2/2)

### • Récapitulatif



## Les RPCs : interface de bas niveau (1/1)

### • Principe

- interface de haut niveau : Uniquement sur UDP, message de moins de 8k, beaucoup d'autres limitations ...
- Davantage de contrôle, de paramétrage et de fonctionnalités :
  - choix du protocole de Transport
  - gestion des structures de données internes, des temporisations, des buffers
  - sécurisation, authentification
  - diffusion
  - asynchronisme...
- Un ensemble de bibliothèques :  
 portmap, rpc\_clnt\_auth, rpc\_clnt\_calls, rpc\_clnt\_calls, rpc\_svc\_create, rpc\_svc\_create, rpc\_svc\_err, rpc\_svc\_reg, rpc\_xdr, secure\_rpc  
 ==> 73 primitives ==> expertise pointue
- Utiliser un compilateur RPC pour générer du code...  
 ==> compilateur **rpcgen**

## RPCGEN (1/4)

### • Principe

- **rpcgen** = Outil logiciel simplifiant la programmation des RPCs par la génération automatique de certaines parties de code nécessaire (compilateur RPC)
- Le compilateur **rpcgen** travaille à partir d'un fichier contenant la spécification de l'interface du programme distant.
- Un langage de spécification d'interface (IDL) est utilisé pour les RPC : **RPCL** (Remote Procedure Call Language) proche du C
- Génération automatique des fichiers (convention de nommage):
  - déclarations communes au client et au serveur
  - stub client
  - squelette serveur
  - routines de conversion xdr
  - makefile
- Puis compilation et édition de liens avec les fichiers contenant le code des service et du client (ossatures minimales pouvant être également générée)

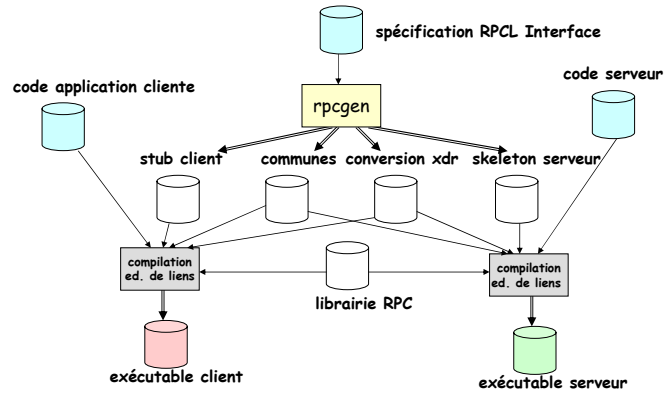
## RPCGEN (2/4)

### • Spécification des interfaces en RPCL

- Le programmeur décrit l'interface dans un fichier RPCL d'extension .x
- Cette spécification par un IDL est indépendante d'un langage de programmation :
  - Traitement de l'hétérogénéité
  - Types de données différents selon les langages (représentation internes différentes, définition de types indépendante de la représentation)
- Description d'une interface :
  - Description des types élémentaires (arguments, résultats, exceptions)
  - Nom des procédures de conversion pour types complexes

### RPCGEN (3/4)

- Illustration : modèle



### RPCGEN (4/4)

- Illustration : instantiation

